

OPERATING SYSTEMS – HANDWRITTEN EXAM NOTES

UNIT 1: Operating System Overview, Dual-Mode Operation, OS Structures, System Calls & Services

B.Tech Mid-1 Examination Preparation | Complete Question Bank Solutions

SECTION A – 2 MARKS QUESTIONS

Question 1

What is an Operating System?

Definition

An Operating System (OS) is system software that acts as an intermediary between the computer user and the computer hardware. It manages hardware resources (CPU, memory, storage, I/O devices) and provides a convenient, efficient environment in which users can execute their application programs.

Two Broad Roles

- **Resource Manager:** Allocates CPU time, memory space, files, and I/O devices fairly among competing user programs.
- **Control Program:** Controls the execution of user programs to prevent errors and improper use of the computer.

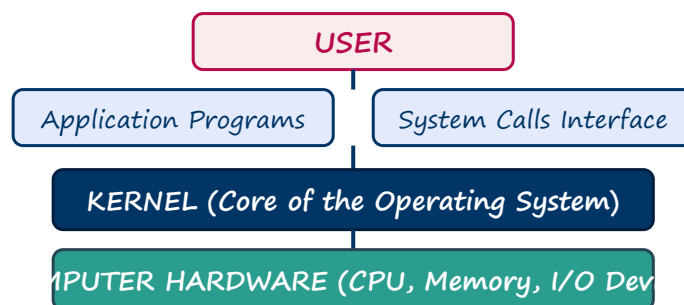


Fig 1.1: Operating System as an Interface between User and Hardware

Question 2

Define Kernel

Definition

The **Kernel** is the core, central part of the Operating System that is loaded into memory first during booting and stays resident in RAM as long as the computer runs. It has direct control over hardware and provides essential process, memory, and device management services to the rest of the system.

Key Features

- It is the first program loaded when the system boots.
- It handles all low-level communication between hardware and software.
- Provides core system services: process scheduling, memory allocation, file systems, and I/O control.

Question 3

What is Multiprogramming?

Definition & Key Points

Multiprogramming is a technique where multiple programs (jobs) reside in main memory simultaneously, and the CPU switches among them to maximize CPU utilization.

- When one executing job waits for an I/O operation to complete, the OS immediately switches the CPU to execute another ready job.
- **Benefit:** Keeps the CPU constantly busy at all times, drastically increasing overall system throughput and eliminating idle CPU time.

Question 4

How does Dual-Mode Operation work?

Concept & Working

To protect the OS and hardware from incorrect or malicious user programs, the CPU provides two hardware execution modes, distinguished by a **Mode Bit** in a status register:

- **User Mode (Mode Bit = 1):** User applications execute with restricted privileges. They cannot directly access hardware or execute privileged instructions.
- **Kernel / Monitor Mode (Mode Bit = 0):** The OS kernel executes with full, unrestricted access to hardware and all privileged CPU instructions.

Mode Transition:

- User → Kernel: A **system call, trap, or interrupt** automatically sets Mode Bit = 0.
- Kernel → User: A **return-from-interrupt** instruction resets Mode Bit = 1.

Question 5

What are the objectives of an Operating System?

1. **Convenience:** Makes the computer hardware easy, pleasant, and intuitive for users to interact with.
2. **Efficiency:** Manages system resources (CPU, RAM, disks) effectively to achieve maximum throughput and performance.
3. **Ability to Evolve:** Permits effective development, testing, and introduction of new features and device drivers without disrupting existing services.

Question 6

What is the use of fork() and exec() system calls?

- **fork():** Creates a new child process that is an exact duplicate of the calling parent process. After fork(), both parent and child execute concurrently from the next instruction.
- **exec():** Replaces the current process's memory space and program image with a completely new program. Typically called inside the child process after fork().
- Together, fork() + exec() forms the standard mechanism in UNIX/Linux for launching new programs.

Question 7

Define Open Source Operating System

Definition & Characteristics

An **Open Source Operating System** is one whose source code is released under a free software license, making it freely available for anyone to view, study, modify, and redistribute.

- **Characteristics:** Publicly accessible source code, no proprietary lock-in, developed collaboratively by global communities.
- **Examples:** Linux (Ubuntu, Fedora, Debian), FreeBSD, OpenSolaris, Android (based on Linux kernel).

Question 8

List the services provided by an Operating System

1. **Services Provided to Users:**
 - **User Interface:** CLI (Command Line) or GUI (Graphical User Interface).

- **Program Execution:** Loading programs into memory and running them.
- **I/O Operations:** Controlled access to input/output devices.
- **File-System Manipulation:** Creating, reading, writing, and deleting files.
- **Communication:** Inter-process communication via shared memory or message passing.
- **Error Detection:** Detecting and handling hardware/software errors.

2. Services for System Efficiency:

- **Resource Allocation:** Sharing CPU, memory, and devices among multiple users.
- **Accounting:** Tracking resource usage for auditing and billing.
- **Protection and Security:** Controlling access to system resources.

Question 9

Define System Calls

Definition & Categories

A **System Call** is the programming interface through which a running user program requests a service directly from the operating system's kernel. It serves as the only legal gateway into kernel mode.

Category	Example System Calls
Process Control	fork(), exec(), exit(), wait()
File Management	open(), read(), write(), close()
Device Management	ioctl(), read(), write()
Information Maintenance	getpid(), alarm(), sleep()
Communication	pipe(), shmget(), mmap()
Protection	chmod(), umask(), chown()

Question 10

What is the purpose of System Programs?

System Programs (System Utilities) provide a convenient environment for program development and execution, allowing users to perform common administrative and maintenance tasks without writing code from scratch.

- **File Management:** cp, mv, rm, ls.
- **Status Information:** date, time, ps, top, disk usage tools.

- **Programming Language Support:** Compilers (gcc), assemblers, debuggers (gdb).
- **Program Loading & Execution:** Loaders, dynamic linkers.
- **Communications:** Web browsers, SSH clients, FTP utilities.

----- SECTION B — 10 MARKS QUESTIONS -----

Essay Question 1

Explain the different functions of an Operating System and discuss the various services provided by an Operating System

1. Introduction & Definition

An Operating System (OS) is system software that acts as an intermediary between the computer user and computer hardware. It manages hardware resources (CPU, memory, I/O devices) and provides an environment for program execution.

2. Functions of an Operating System

1. **Process Management:** Creates, schedules, and terminates processes; maintains Process Control Blocks (PCBs) and manages context switching.
2. **Memory Management:** Tracks each byte in main memory, dynamically allocates and deallocates memory spaces, and manages virtual memory via paging and segmentation.
3. **File System Management:** Organizes files into directories, manages access permissions, and handles file creation, reading, writing, and deletion.
4. **I/O Device Management:** Hides device-specific hardware complexities using device drivers, buffers, spooling, and caching.
5. **Security & Protection:** Controls user authentication (passwords) and enforces access control matrices to prevent unauthorized resource access.
6. **Networking:** Manages network interfaces and communication protocols (TCP/IP) for distributed data exchange.

3. Services Provided by the Operating System

A. Services for Users:

- **User Interface:** CLI (shell) or GUI (desktop) for interacting with the system.
- **Program Execution:** Loads programs into memory and executes them to completion.
- **I/O Operations:** Provides safe, uniform access to I/O hardware.
- **File-System Manipulation:** Manages directories and access rights.
- **Communication:** Inter-process communication via shared memory or message passing.
- **Error Detection:** Continuously monitors CPU, memory, and devices for errors.

B. Services for System Efficiency:

- **Resource Allocation:** Distributes CPU cycles, memory, and devices fairly among multiple users.
- **Accounting:** Records resource usage metrics for billing and performance auditing.
- **Protection & Security:** Guarantees safe, isolated execution of all processes.

Conclusion: The OS forms the foundational software layer of modern computing, turning raw hardware into a safe, multi-user platform.

Essay Question 2

Explain Operating System Structures

1. Structural Approaches in OS Design

Operating systems employ different structural architectures to balance modularity, performance, and reliability:

2. Classification of OS Structures

(a) Simple (Monolithic) Structure

- All OS functionality (process scheduling, memory management, file systems, device drivers) is compiled into a single large binary running in kernel mode.
- **Advantage:** Extremely fast execution due to direct internal function calls without context switches.
- **Disadvantage:** A single bug or crash in any driver can bring down the entire operating system (e.g., MS-DOS, traditional UNIX, Linux).

(b) Layered Approach

- The OS is broken into numbered hierarchical layers. **Layer 0** is the hardware, and **Layer N** is the user interface.
- Each layer can invoke services only from the layer immediately below it ($N-1$).
- **Advantage:** Modularity, simplicity of debugging and verification.
- **Disadvantage:** Careful layer ordering is difficult, and layer-to-layer call overhead reduces speed.

(c) Microkernel Structure

- Removes all non-essential components from the kernel and runs them as user-space server processes (file system server, device drivers).
- The microkernel retains only minimal IPC, basic memory management, and low-level scheduling.

- **Advantage:** High reliability and security; if a file server crashes, the kernel remains unharmed. Easy to extend.
- **Disadvantage:** Message passing (IPC) overhead between user servers and kernel degrades performance (e.g., QNX, Mach, Minix).

(d) Modular Structure (Loadable Kernel Modules - LKM)

- The kernel maintains a core set of components and dynamically loads or unloads module drivers (e.g., network drivers, file systems) at runtime or boot time.
- Combines monolithic performance with microkernel flexibility (e.g., Modern Linux, Solaris).

(e) Hybrid Systems

- Combines multiple structures to achieve optimal balance of performance, security, and modularity (e.g., Windows NT, macOS / iOS Darwin kernel).

3. Comparison: Monolithic Kernel vs Microkernel

Basis	Monolithic Kernel	Microkernel
Structure	Single large block of code in kernel space	Minimal core + user-space servers
Communication	Direct function calls (fast)	Message Passing / IPC (adds overhead)
Speed	Faster (no IPC overhead)	Comparatively slower
Reliability	A single bug can crash the whole OS	High reliability; isolated faults
Extensibility	Difficult to modify and extend	Easy to add new user services
Examples	Linux, traditional UNIX	QNX, Mach, Minix

Essay Question 3

Explain about the Dual-Mode Operation in OS with a neat block diagram

1. Concept of Dual-Mode Operation

Dual-mode operation is a hardware-supported protection architecture that prevents user programs from directly manipulating hardware or executing privileged CPU instructions.

2. Execution Modes

- **User Mode (Mode Bit = 1):** Execution is restricted. Programs cannot execute privileged instructions (such as disabling interrupts or modifying page tables) or directly perform I/O operations.

- **Kernel / Monitor Mode (Mode Bit = 0):** The OS runs with full hardware privileges. All instructions and memory regions can be accessed.

3. Mode Transition Sequence

1. System boots and starts in **Kernel Mode** (Mode Bit = 0).
2. OS loads and transfers control to a user application, resetting **Mode Bit = 1**.
3. User process executes; when it requires an OS service (e.g., reading a file), it executes a **System Call / Trap**, which automatically switches **Mode Bit = 0**.
4. Control passes to the kernel routine, which validates parameters and executes the privileged operation.
5. Before returning control to the user application, the **return-from-interrupt** instruction resets **Mode Bit = 1**.

Importance: Dual-mode operation protects the OS from errant or malicious programs, ensuring that no user application can seize control or corrupt system memory.

Essay Question 4

What is Operating System? Explain Computing Environments

1. Definition of Operating System

An Operating System is system software that manages computer hardware resources and acts as an interface between the user and hardware.

2. Major Computing Environments

1. **Traditional Computing:** Standard standalone desktop PCs and workstations connected via a local network, running office applications and web browsers.
2. **Mobile Computing:** Handheld portable smartphones and tablets (Android, iOS) featuring touch interfaces, GPS, wireless radios, and power-aware OS management.
3. **Distributed / Client-Server Computing:** Multiple independent computing nodes interconnected over a network to share computing load and storage (clients make requests, centralized servers fulfill them).
4. **Peer-to-Peer (P2P) Computing:** Decentralized network where every node acts simultaneously as both a client and a server, sharing resources directly (e.g., BitTorrent).
5. **Virtualization:** Software abstraction (Hypervisors like VMware, VirtualBox) that divides physical hardware into multiple isolated virtual machines (VMs) running guest OSes.
6. **Cloud Computing:** On-demand delivery of compute, storage, and software services over the Internet, classified as:
 - **IaaS (Infrastructure as a Service):** Virtual servers, storage (AWS EC2).
 - **PaaS (Platform as a Service):** Development frameworks (Google App Engine).

- SaaS (Software as a Service): End-user cloud applications (Google Docs, Office 365).
7. **Real-Time Embedded Systems:** Microcontrollers embedded within cars, medical devices, and robots governed by strict deterministic timing deadlines (Hard vs Soft Real-Time).

Essay Question 5

Write the differences between Monolithic Kernel and Microkernel

Basis	Monolithic Kernel	Microkernel
Structure	Single large address space containing all OS services	Small minimal kernel + separate user-space servers
Communication	Direct function calls within kernel space	Message Passing (IPC) between user servers
Execution Speed	Faster (zero IPC context-switch overhead)	Slower due to repeated IPC message switching
Fault Isolation & Reliability	Low (a crash in one driver crashes the entire system)	High (crashed user server can be restarted independently)
Extensibility	Difficult (requires kernel recompilation)	Easy (add new user-level servers without touching kernel)
Security	Less secure (large attack surface in kernel mode)	More secure (minimal code in privileged mode)
Memory Footprint	Requires larger contiguous memory in kernel space	Small kernel footprint in memory
Examples	Linux, traditional UNIX, MS-DOS	QNX, Mach, Minix

Essay Question 6

Explain briefly System Calls with examples

1. Definition & Working

A **System Call** is the programmatic interface provided by the OS kernel that enables user applications to request privileged operations.

Step-by-Step Execution:

1. User program calls an API / library wrapper function (e.g., `printf()` or `open()`).
2. The library issues a software interrupt / trap into the kernel.
3. Hardware switches CPU from User Mode to Kernel Mode (Mode Bit \$1 \rightarrow 0\$).
4. Kernel checks the system call number in the **System Call Table** and executes the service routine.

5. Control and return values are passed back to the user program in User Mode (Mode Bit \$0 \rightarrow 1\$).

2. The 6 Categories of System Calls

- **Process Control:** `fork()` (create process), `exec()` (load program), `exit()` (terminate), `wait()`.
- **File Management:** `open()`, `read()`, `write()`, `close()`, `unlink()`.
- **Device Management:** `ioctl()`, `read()`, `write()`.
- **Information Maintenance:** `getpid()`, `alarm()`, `sleep()`, `time()`.
- **Communication:** `pipe()`, `shmget()`, `mmap()`, `socket()`.
- **Protection:** `chmod()`, `umask()`, `chown()`.

Essay Question 7

Explain briefly User and Operating System Interface

1. The Three Primary User Interfaces

1. **Command Line Interface (CLI):** Also called a shell or command interpreter. Users type text commands directly (e.g., `bash` in Linux, `Command Prompt/PowerShell` in Windows). Highly efficient, fast, scriptable, and low resource overhead.
2. **Graphical User Interface (GUI):** Uses windows, icons, menus, and pointer devices (WIMP interface). User-friendly and intuitive for all skill levels, but consumes more RAM and CPU cycles (e.g., `Windows Desktop`, `macOS Aqua`, `GNOME`).
3. **Touch-Screen Interface:** Used on mobile smartphones and tablets. Relies on physical gestures (tap, swipe, pinch-to-zoom) and on-screen virtual keyboards (e.g., `Android`, `iOS`).

2. Interface Comparison Table

Interface	Advantages	Disadvantages
CLI	Fast for experienced users; scriptable; low resource usage	Requires memorizing commands; steep learning curve
GUI	Intuitive, visual, easy for beginners	More resource-intensive; less suited for automated scripting
Touch-Screen	Natural, gesture-based mobility	Lower precision compared to physical mouse/keyboard

Essay Question 8

Discuss about Operating System Services

1. Services for the User

- **User Interface:** Provides CLI, GUI, or touch interface.
- **Program Execution:** Loads code into RAM, initializes memory, and manages runtime execution.
- **I/O Operations:** Handles all device communications since users cannot perform I/O directly.
- **File-System Manipulation:** Manages files, directories, read/write permissions, and disk blocks.
- **Communications:** Inter-process communication via shared memory or message passing.
- **Error Detection:** Catches hardware faults (parity errors), CPU exceptions (divide by zero), and program bugs.

2. Services for System Operation & Efficiency

- **Resource Allocation:** Dynamically allocates CPU cores, memory pages, and disk bandwidth.
- **Accounting:** Maintains statistics on CPU and memory consumption per user.
- **Protection & Security:** Enforces user authentication and access control rules.

Essay Question 9

Write a short notes on: (a) Operating System Design & Implementation (b) System Programs

(a) Operating System Design and Implementation

- **Design Goals:**
 - *User Goals:* Convenient, reliable, safe, fast, and easy to learn.
 - *System Goals:* Easy to design, implement, debug, flexible, and efficient.
- **Mechanism vs Policy:**
 - *Mechanism:* Determines **how** to do something (e.g., timer hardware for context switching).
 - *Policy:* Determines **what** will be done (e.g., how long the timer quantum should be).
 - **Principle:** Separating policy from mechanism ensures modularity and easy future reconfigurability!
- **Implementation Languages:** Early OSes were in Assembly; modern OSes are written in C/C++ with minimal Assembly for hardware boot loaders to ensure portability across architectures.

(b) System Programs (System Utilities)

Category	Examples
File Management	Create, delete, copy, rename, list files (cp, rm, ls)

Status Information	Date, time, memory/disk usage, logging (top, df)
File Modification	Text editors (vim, nano), search tools (grep)
Programming Language Support	Compilers (gcc), assemblers, debuggers (gdb)
Program Loading & Execution	Loaders, linkers, overlay managers
Communications	Browsers, SSH clients, remote login tools

Essay Question 10

Discuss about Operating System Debugging

1. Definition & Scope

OS Debugging is the process of detecting, isolating, and fixing hardware and software errors (bugs) and eliminating performance bottlenecks within an operating system.

2. Key Debugging Techniques & Tools

Technique	Purpose & Mechanism
Failure Analysis & Core Dump	When a process crashes, its memory state is written to a Core Dump file for post-mortem analysis.
Crash Dump Analysis	When the OS kernel panics/crashes, kernel memory is saved to a Crash Dump file analyzed via kernel debuggers.
Trace Listings	Records every system call executed by a running program (e.g., strace in Linux, truss in Solaris).
System Monitoring Tools	Displays real-time CPU, RAM, and disk utilization (e.g., Task Manager, top, vmstat).
Dynamic Probing (DTrace)	Dynamically inserts instrumentation probes into live production kernels without requiring code recompilation.
Performance Tuning	Profiles code execution hotspots and eliminates I/O or memory bottlenecks.

----- LAST-MINUTE EXAM REVISION (UNIT 1) -----

Quick Master Summary Table

Topic	Core Exam Keyword	Key Takeaway
Operating System	Resource Manager & Control Program	Intermediary between user and computer hardware

Kernel	<i>Resident Core</i>	<i>First program loaded at boot; manages CPU, RAM, and I/O</i>
Dual-Mode	<i>User (1) vs Kernel (0) Mode</i>	<i>Traps switch \$1 \rightarrow 0\$; protects OS from rogue user code</i>
System Calls	<i>Kernel API Gateway</i>	<i>6 categories: Process, File, Device, Info, Comms, Protection</i>
Monolithic vs Microkernel	<i>Function calls vs Message Passing</i>	<i>Monolithic is faster; Microkernel is more modular & fault-tolerant</i>